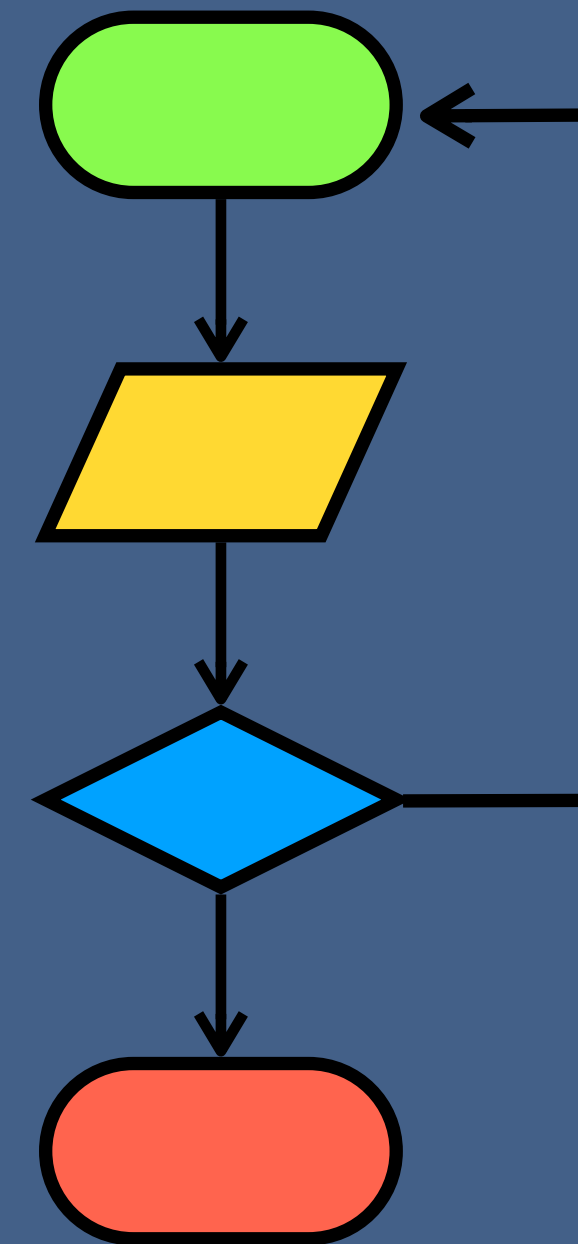


Dynamic Programming



Dynamic Programming - Number Factor

Problem Statement:

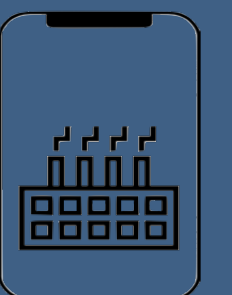
Given N, find the number of ways to express N as a sum of 1, 3 and 4.

Example 1

- $N = 4$
- Number of ways = 4
- Explanation : There are 4 ways we can express N. $\{4\}, \{1,3\}, \{3,1\}, \{1,1,1,1\}$

Example 2

- $N = 5$
- Number of ways = 6
- Explanation : There are 6 ways we can express N. $\{4,1\}, \{1,4\}, \{1,3,1\}, \{3,1,1\}, \{1,1,3\}, \{1,1,1,1,1\}$

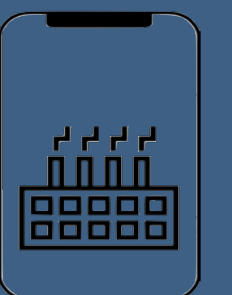


Dynamic Programming - Number Factor

Problem Statement:

Given N, find the number of ways to express N as a sum of 1, 3 and 4.

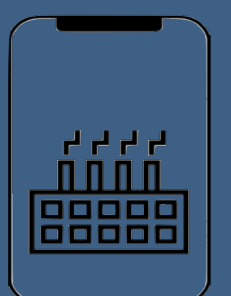
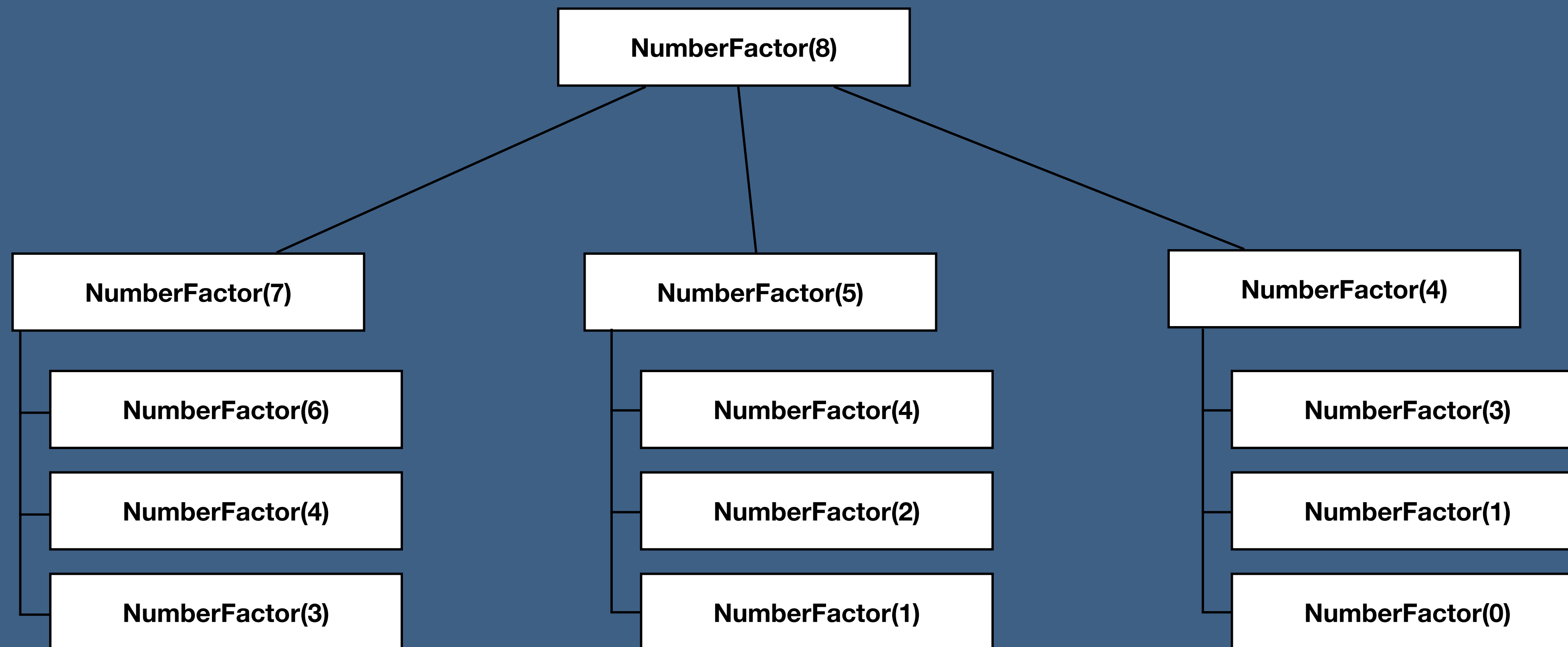
```
NumberFactor(N)
  If N in (0,1,2) return 1
  If N = 3 return 2
  Else
    return NumberFactor(N-1) + NumberFactor(N-3) + NumberFactor(N-4)
```



Dynamic Programming - Number Factor

Problem Statement:

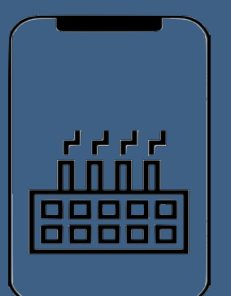
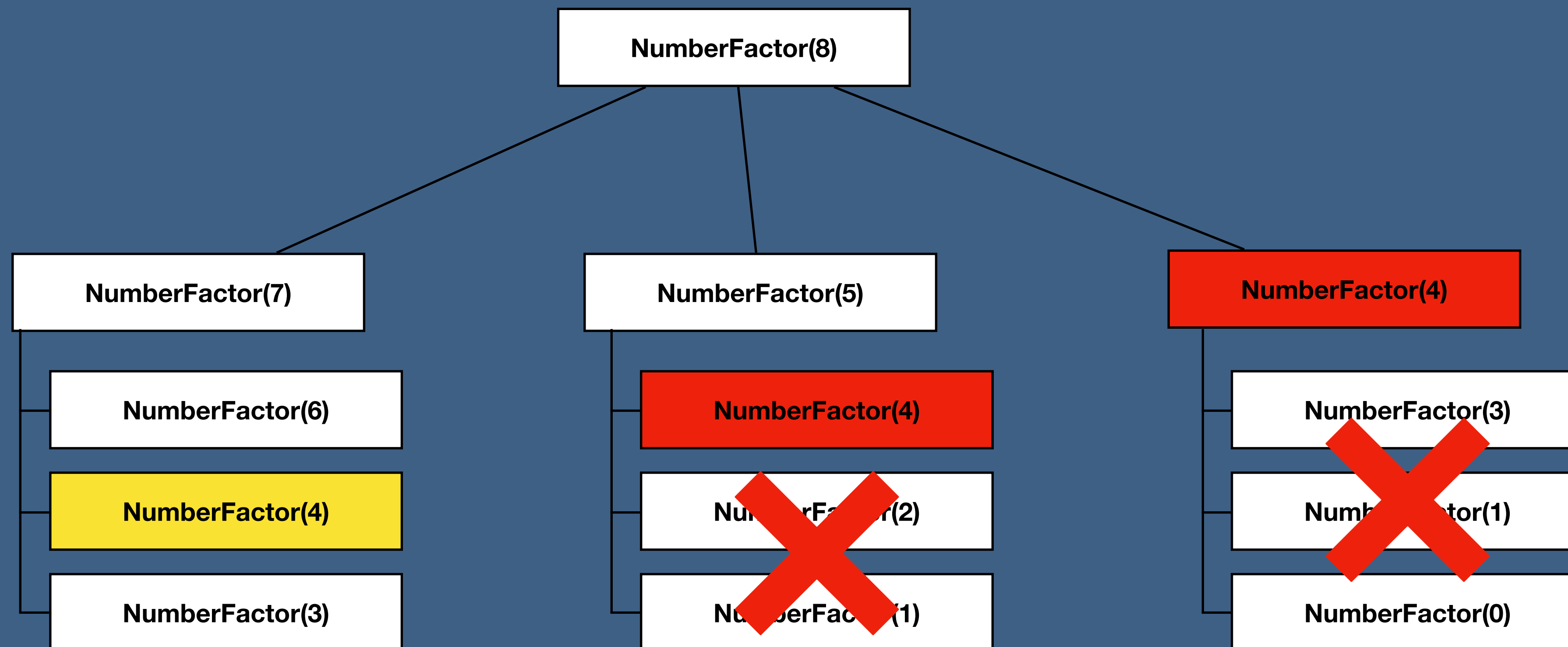
Given N, find the number of ways to express N as a sum of 1, 3 and 4.



Dynamic Programming - Number Factor

Problem Statement:

Given N, find the number of ways to express N as a sum of 1, 3 and 4.



Dynamic Programming - Number Factor

Problem Statement:

Given N, find the number of ways to express N as a sum of 1, 3 and 4.

```
NumberFactor(N):
```

```
    If N in (0,1,2) return 1
```

```
    If N = 3 return 2
```

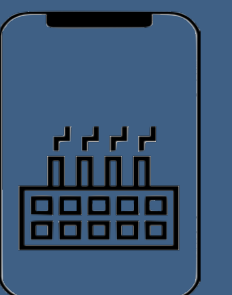
```
    Else
```

```
        rec1 = NumberFactor(N-1)
```

```
        rec2 = NumberFactor(N-3)
```

```
        rec3 = NumberFactor(N-4)
```

```
    return rec1 + rec2 + rec3
```

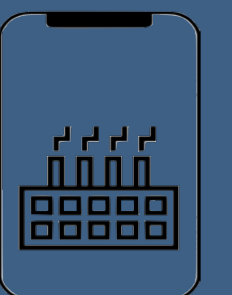


Dynamic Programming - Number Factor

Problem Statement:

Given N, find the number of ways to express N as a sum of 1, 3 and 4.

```
NumberFactor(N, dp):  
  
    If N in (0,1,2) return 1  
    If N = 3 return 2  
  
    Else  
        rec1 = NumberFactor(N-1)  
        rec2 = NumberFactor(N-3)  
        rec3 = NumberFactor(N-4)  
  
        return rec1 + rec2 + rec3
```

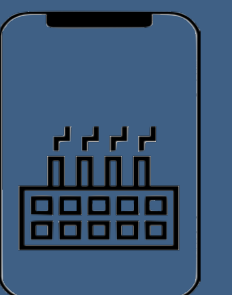


Dynamic Programming - Number Factor

Problem Statement:

Given N, find the number of ways to express N as a sum of 1, 3 and 4.

```
NumberFactor(N, dp):  
  
    If N in (0,1,2) return 1  
    If N = 3 return 2  
    Elif N in dp return dp[N]  
    Else  
        rec1 = NumberFactor(N-1)  
        rec2 = NumberFactor(N-3)  
        rec3 = NumberFactor(N-4)  
  
        return rec1 + rec2 + rec3
```

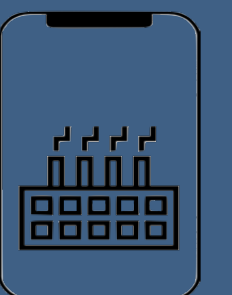


Dynamic Programming - Number Factor

Problem Statement:

Given N, find the number of ways to express N as a sum of 1, 3 and 4.

```
NumberFactor(N, dp):  
  
    If N in (0,1,2) return 1  
    If N = 3 return 2  
    Elif N in dp return dp[N]  
    Else  
        rec1 = NumberFactor(N-1)  
        rec2 = NumberFactor(N-3)  
        rec3 = NumberFactor(N-4)  
        dp[N] = rec1 + rec2 + rec3
```

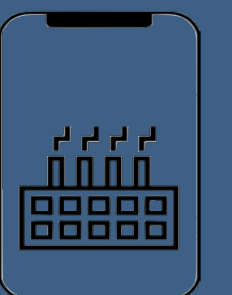


Dynamic Programming - Number Factor

Problem Statement:

Given N, find the number of ways to express N as a sum of 1, 3 and 4.

```
NumberFactor(N, dp):  
  
    If N in (0,1,2) return 1  
    If N = 3 return 2  
    Elif N in dp return dp[N]  
    Else  
        rec1 = NumberFactor(N-1)  
        rec2 = NumberFactor(N-3)  
        rec3 = NumberFactor(N-4)  
        dp[N] = rec1 + rec2 + rec3  
        return dp[N]
```

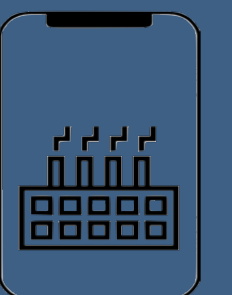


Dynamic Programming - Number Factor

Problem Statement:

Given N, find the number of ways to express N as a sum of 1, 3 and 4.

```
NumberFactor(N, dp): → Step 1  
  
    If N in (0,1,2) return 1  
    If N = 3 return 2  
    Elif N in dp return dp[N] → Step 2  
    Else  
        rec1 = NumberFactor(N-1)  
        rec2 = NumberFactor(N-3)  
        rec3 = NumberFactor(N-4)  
        dp[N] = rec1 + rec2 + rec3 → Step 3  
        return dp[N] → Step 4
```



Dynamic Programming - Number Factor

Problem Statement:

Given N, find the number of ways to express N as a sum of 1, 3 and 4.

Top Down Approach

NF(0)	NF(1)	NF(2)	NF(3)	NF(4)	NF(5)	NF(6)	NF(7)
1	1	1	2				NF6+NF4+NF3

NF(0)	NF(1)	NF(2)	NF(3)	NF(4)	NF(5)	NF(6)	NF(7)
1	1	1	2			NF5+NF3+NF2	NF6+NF4+NF3

NF(0)	NF(1)	NF(2)	NF(3)	NF(4)	NF(5)	NF(6)	NF(7)
1	1	1	2		NF4+NF2+NF1	NF5+NF3+NF2	NF6+NF4+NF3

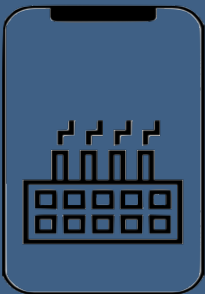
NF(0)	NF(1)	NF(2)	NF(3)	NF(4)	NF(5)	NF(6)	NF(7)
1	1	1	2	NF3+NF2+NF0	NF4+NF2+NF1	NF5+NF3+NF2	NF6+NF4+NF3

NF(0)	NF(1)	NF(2)	NF(3)	NF(4)	NF(5)	NF(6)	NF(7)
1	1	1	2	2+1+1=4	NF4+NF2+NF1	NF5+NF3+NF2	NF6+NF4+NF3

NF(0)	NF(1)	NF(2)	NF(3)	NF(4)	NF(5)	NF(6)	NF(7)
1	1	1	2	2+1+1=4	4+1+1=6	NF5+NF3+NF2	NF6+NF4+NF3

NF(0)	NF(1)	NF(2)	NF(3)	NF(4)	NF(5)	NF(6)	NF(7)
1	1	1	2	2+1+1=4	4+1+1=6	6+2+1=9	NF6+NF4+NF3

NF(0)	NF(1)	NF(2)	NF(3)	NF(4)	NF(5)	NF(6)	NF(7)
1	1	1	2	2+1+1=4	4+1+1=6	6+2+1=9	9+4+2=15



Dynamic Programming - Number Factor

Problem Statement:

Given N, find the number of ways to express N as a sum of 1, 3 and 4.

Bottom Up Approach

NF(0)	NF(1)	NF(2)	NF(3)	NF(4)	NF(5)	NF(6)	NF(7)
1	1	1	2	NF3+NF1+NF0			

NF(0)	NF(1)	NF(2)	NF(3)	NF(4)	NF(5)	NF(6)	NF(7)
1	1	1	2	NF3+NF1+NF0	NF4+NF2+NF1		

NF(0)	NF(1)	NF(2)	NF(3)	NF(4)	NF(5)	NF(6)	NF(7)
1	1	1	2	NF3+NF1+NF0	NF4+NF2+NF1	NF6+NF3+NF2	

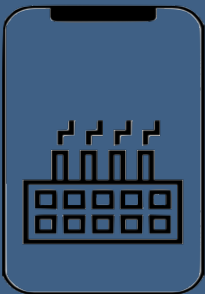
NF(0)	NF(1)	NF(2)	NF(3)	NF(4)	NF(5)	NF(6)	NF(7)
1	1	1	2	NF3+NF1+NF0	NF4+NF2+NF1	NF6+NF3+NF2	NF6+NF4+NF3

NF(0)	NF(1)	NF(2)	NF(3)	NF(4)	NF(5)	NF(6)	NF(7)
1	1	1	2	2+1+1=4	NF4+NF2+NF1	NF6+NF3+NF2	NF6+NF4+NF3

NF(0)	NF(1)	NF(2)	NF(3)	NF(4)	NF(5)	NF(6)	NF(7)
1	1	1	2	2+1+1=4	4+1+1=6	NF6+NF3+NF2	NF6+NF4+NF3

NF(0)	NF(1)	NF(2)	NF(3)	NF(4)	NF(5)	NF(6)	NF(7)
1	1	1	2	2+1+1=4	4+1+1=6	6+2+1=9	NF6+NF4+NF3

NF(0)	NF(1)	NF(2)	NF(3)	NF(4)	NF(5)	NF(6)	NF(7)
1	1	1	2	2+1+1=4	4+1+1=6	6+2+1=9	9+4+2=15



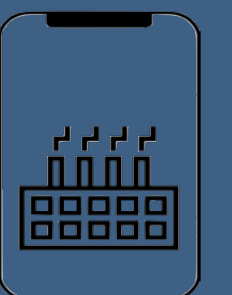
Dynamic Programming - Number Factor

Problem Statement:

Given N, find the number of ways to express N as a sum of 1, 3 and 4.

Bottom Up Approach

```
numberFactor(n)
    tb = {1,1,1,2}
    for i in range(4, n+1):
        tb.append(tb[i-1]+tb[i-3]+tb[i-4])
    return tb[n]
```



Dynamic Programming - House Robber

Problem Statement:

- Given N number of houses along the street with some amount of money
- Adjacent houses cannot be stolen
- Find the maximum amount that can be stolen

Example 1



Answer

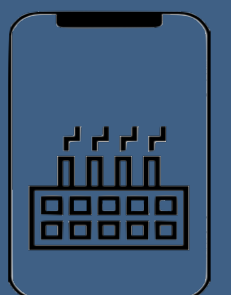
- Maximum amount = 41
- Houses that are stolen : 7, 30, 4

$$\text{Option1} = 6 + f(5)$$



$$\text{Max}(\text{Option1}, \text{Option2})$$

$$\text{Option2} = 0 + f(6)$$

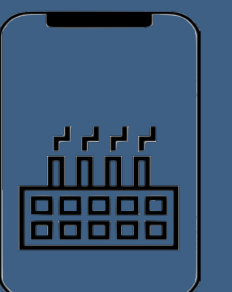


Dynamic Programming - House Robber

Problem Statement:

- Given N number of houses along the street with some amount of money
- Adjacent houses cannot be stolen
- Find the maximum amount that can be stolen

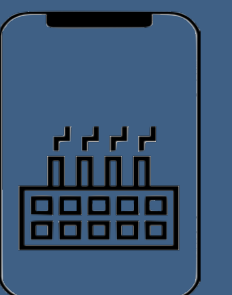
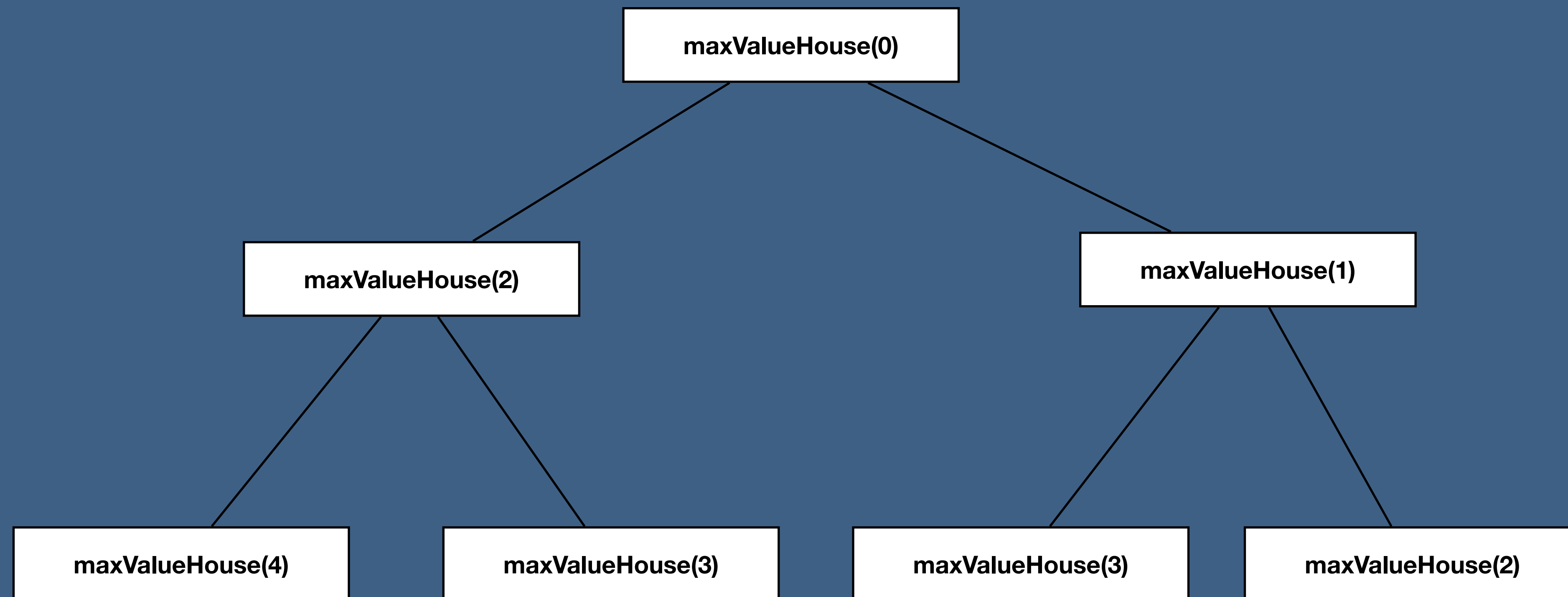
```
maxValueHouse(houses, currentHouse)
    If currentHouse > length of houses
        return 0
    Else
        stealFirstHouse = currentHouse + maxValueHouse(houses, currentHouse+2)
        skipFirstHouse = maxValueHouse(houses, currentHouse+1)
        return max(stealFirstHouse, skipFirstHouse)
```



Dynamic Programming - House Robber

Problem Statement:

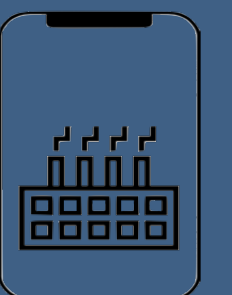
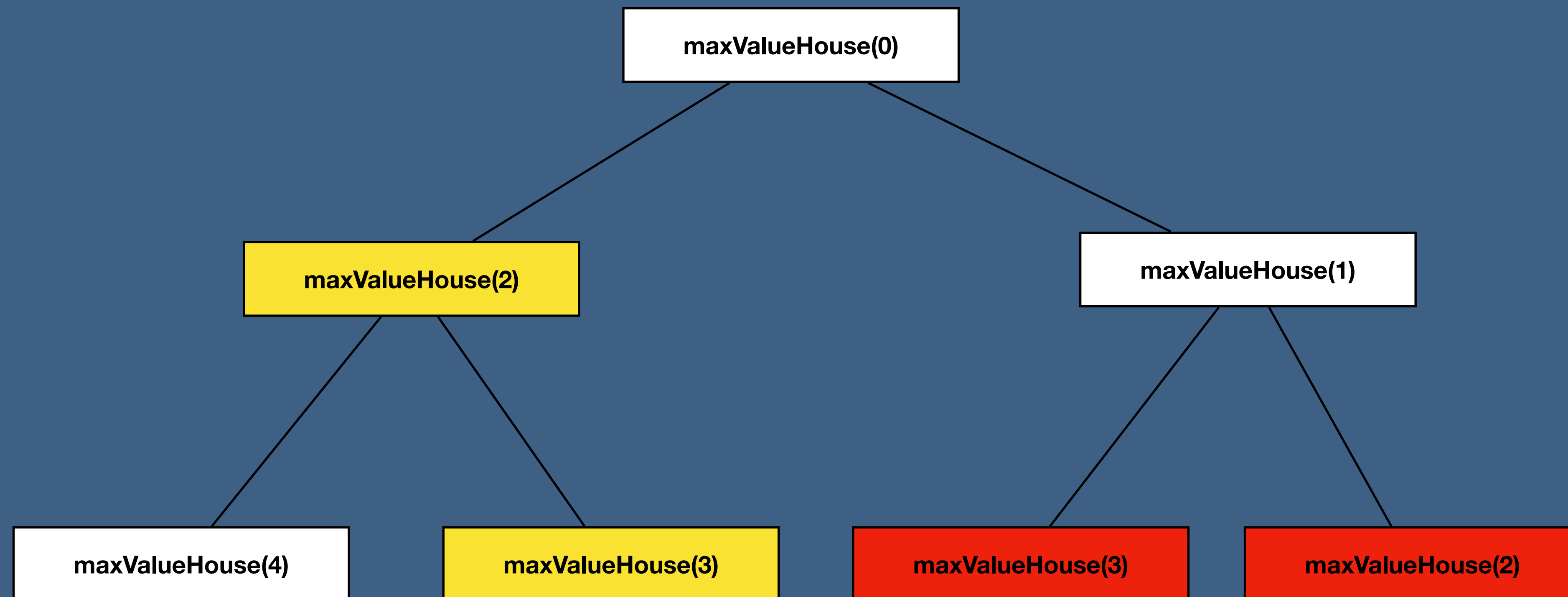
- Given N number of houses along the street with some amount of money
- Adjacent houses cannot be stolen
- Find the maximum amount that can be stolen



Dynamic Programming - House Robber

Problem Statement:

- Given N number of houses along the street with some amount of money
- Adjacent houses cannot be stolen
- Find the maximum amount that can be stolen

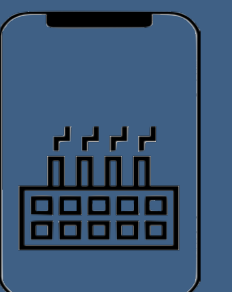


Dynamic Programming - House Robber

Problem Statement:

- Given N number of houses along the street with some amount of money
- Adjacent houses cannot be stolen
- Find the maximum amount that can be stolen

```
maxValueHouse(houses, currentHouse)
  If currentHouse > length of houses
    return 0
  Else
    stealFirstHouse = currentHouse + maxValueHouse(houses, currentHouse+2)
    skipFirstHouse = maxValueHouse(houses, currentHouse+1)
    return max(stealFirstHouse, skipFirstHouse)
```

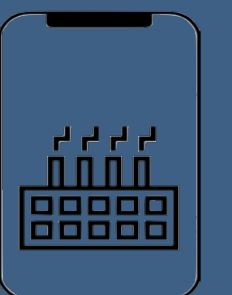


Dynamic Programming - House Robber

Problem Statement:

- Given N number of houses along the street with some amount of money
- Adjacent houses cannot be stolen
- Find the maximum amount that can be stolen

```
maxValueHouse(houses, currentHouse, dp): dp → Step 1  
    If currentHouse > length of houses  
        return 0  
    Else  
        stealFirstHouse = currentHouse + maxValueHouse(houses, currentHouse+2)  
        skipFirstHouse = maxValueHouse(houses, currentHouse+1)  
        return max(stealFirstHouse, skipFirstHouse)
```

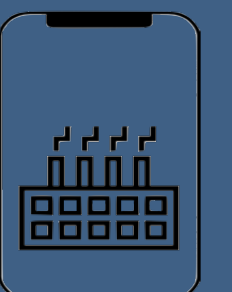


Dynamic Programming - House Robber

Problem Statement:

- Given N number of houses along the street with some amount of money
- Adjacent houses cannot be stolen
- Find the maximum amount that can be stolen

```
maxValueHouse(houses, currentHouse, dp):  
    If currentHouse > length of houses  
        return 0  
    Else  
        If currentHouse not in dp: → Step 2  
            stealFirstHouse = currentHouse + maxValueHouse(houses, currentHouse+2)  
            skipFirstHouse = maxValueHouse(houses, currentHouse+1)  
            return max(stealFirstHouse, skipFirstHouse)  
        → Step 1
```

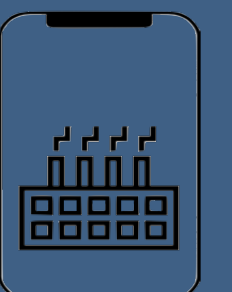


Dynamic Programming - House Robber

Problem Statement:

- Given N number of houses along the street with some amount of money
- Adjacent houses cannot be stolen
- Find the maximum amount that can be stolen

```
maxValueHouse(houses, currentHouse, dp):  
    If currentHouse > length of houses  
        return 0  
    Else  
        If currentHouse not in dp: —————→ Step 2  
            stealFirstHouse = currentHouse + maxValueHouse(houses, currentHouse+2)  
            skipFirstHouse = maxValueHouse(houses, currentHouse+1)  
            dp[currentHouse] = max(stealFirstHouse, skipFirstHouse) —————→ Step 3  
        return dp[currentHouse] —————→ Step 4
```



Dynamic Programming - House Robber

Top Down Approach

Problem Statement:

- Given N number of houses along the street with some amount of money
- Adjacent houses cannot be stolen
- Find the maximum amount that can be stolen

H0	H1	H2	H3	H4	H5	H6
6	7	1	30	8	2	4

HR(0)	HR(1)	HR(2)	HR(3)	HR(4)	HR(5)	HR(6)
max(H0+HR2, HR1)						

HR(0)	HR(1)	HR(2)	HR(3)	HR(4)	HR(5)	HR(6)
max(H0+HR2, HR1)	max(H1+HR3, HR2)					

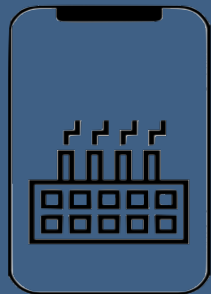
HR(0)	HR(1)	HR(2)	HR(3)	HR(4)	HR(5)	HR(6)
max(H0+HR2, HR1)	max(H1+HR3, HR2)	max(H2+HR4, HR3)				

HR(0)	HR(1)	HR(2)	HR(3)	HR(4)	HR(5)	HR(6)
max(H0+HR2, HR1)	max(H1+HR3, HR2)	max(H2+HR4, HR3)	max(H3+HR5, HR4)			

HR(0)	HR(1)	HR(2)	HR(3)	HR(4)	HR(5)	HR(6)
max(H0+HR2, HR1)	max(H1+HR3, HR2)	max(H2+HR4, HR3)	max(H3+HR5, HR4)	max(H4+HR6, HR5)		

HR(0)	HR(1)	HR(2)	HR(3)	HR(4)	HR(5)	HR(6)
max(H0+HR2, HR1)	max(H1+HR3, HR2)	max(H2+HR4, HR3)	max(H3+HR5, HR4)	max(H4+HR6, HR5)	max(H5+HR7, HR6)	

HR(0)	HR(1)	HR(2)	HR(3)	HR(4)	HR(5)	HR(6)
max(H0+HR2, HR1)	max(H1+HR3, HR2)	max(H2+HR4, HR3)	max(H3+HR5, HR4)	max(H4+HR6, HR5)	max(H5+HR7, HR6)	max(H6+HR8, HR7)



Dynamic Programming - House Robber

Top Down Approach

Problem Statement:

- Given N number of houses along the street with some amount of money
- Adjacent houses cannot be stolen
- Find the maximum amount that can be stolen

H0	H1	H2	H3	H4	H5	H6
6	7	1	30	8	2	4

HR(0)	HR(1)	HR(2)	HR(3)	HR(4)	HR(5)	HR(6)
max(H0+HR2, HR1)	max(H1+HR3, HR2)	max(H2+HR4, HR3)	max(H3+HR5, HR4)	max(H4+HR6, HR5)	max(H5+HR7, HR6)	max(H6+HR8, HR7)

HR(0)	HR(1)	HR(2)	HR(3)	HR(4)	HR(5)	HR(6)
max(H0+HR2, HR1)	max(H1+HR3, HR2)	max(H2+HR4, HR3)	max(H3+HR5, HR4)	max(H4+HR6, HR5)	max(H5+HR7, HR6)	4

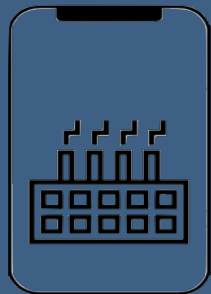
HR(0)	HR(1)	HR(2)	HR(3)	HR(4)	HR(5)	HR(6)
max(H0+HR2, HR1)	max(H1+HR3, HR2)	max(H2+HR4, HR3)	max(H3+HR5, HR4)	max(H4+HR6, HR5)	max(2+0, 4)=4	4

HR(0)	HR(1)	HR(2)	HR(3)	HR(4)	HR(5)	HR(6)
max(H0+HR2, HR1)	max(H1+HR3, HR2)	max(H2+HR4, HR3)	max(H3+HR5, HR4)	max(8+4,4)=12	4	4

HR(0)	HR(1)	HR(2)	HR(3)	HR(4)	HR(5)	HR(6)
max(H0+HR2, HR1)	max(H1+HR3, HR2)	max(H2+HR4, HR3)	max(H3+HR5, HR4)	12	4	4

HR(0)	HR(1)	HR(2)	HR(3)	HR(4)	HR(5)	HR(6)
max(H0+HR2, HR1)	max(H1+HR3, HR2)	max(H2+HR4, HR3)	34	12	4	4

HR(0)	HR(1)	HR(2)	HR(3)	HR(4)	HR(5)	HR(6)
41	41	34	34	12	4	4



Dynamic Programming - House Robber

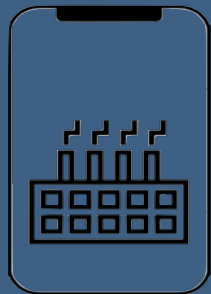
Bottom Up Approach

Problem Statement:

- Given N number of houses along the street with some amount of money
- Adjacent houses cannot be stolen
- Find the maximum amount that can be stolen

H0	H1	H2	H3	H4	H5	H6
6	7	1	30	8	2	4

HR(0)	HR(1)	HR(2)	HR(3)	HR(4)	HR(5)	HR(6)	HR(7)	HR(8)
max(H0+HR2, HR1)	max(H1+HR3, HR2)	max(H2+HR4, HR3)	max(H3+HR5, HR4)	max(H4+HR6, HR5)	max(H5+HR7, HR6)	max(H6+HR8, HR7)	0	0
HR(0)	HR(1)	HR(2)	HR(3)	HR(4)	HR(5)	HR(6)	HR(7)	HR(8)
max(H0+HR2, HR1)	max(H1+HR3, HR2)	max(H2+HR4, HR3)	max(H3+HR5, HR4)	max(H4+HR6, HR5)	max(H5+HR7, HR6)	4	0	0
HR(0)	HR(1)	HR(2)	HR(3)	HR(4)	HR(5)	HR(6)	HR(7)	HR(8)
max(H0+HR2, HR1)	max(H1+HR3, HR2)	max(H2+HR4, HR3)	max(H3+HR5, HR4)	max(H4+HR6, HR5)	4	4	0	0
HR(0)	HR(1)	HR(2)	HR(3)	HR(4)	HR(5)	HR(6)	HR(7)	HR(8)
max(H0+HR2, HR1)	max(H1+HR3, HR2)	max(H2+HR4, HR3)	max(H3+HR5, HR4)	12	4	4	0	0
HR(0)	HR(1)	HR(2)	HR(3)	HR(4)	HR(5)	HR(6)	HR(7)	HR(8)
max(H0+HR2, HR1)	max(H1+HR3, HR2)	max(H2+HR4, HR3)	34	12	4	4	0	0
HR(0)	HR(1)	HR(2)	HR(3)	HR(4)	HR(5)	HR(6)	HR(7)	HR(8)
max(H0+HR2, HR1)	max(H1+HR3, HR2)	34	34	12	4	4	0	0
HR(0)	HR(1)	HR(2)	HR(3)	HR(4)	HR(5)	HR(6)	HR(7)	HR(8)
max(H0+HR2, HR1)	41	34	34	12	4	4	0	0
HR(0)	HR(1)	HR(2)	HR(3)	HR(4)	HR(5)	HR(6)	HR(7)	HR(8)
41	41	34	34	12	4	4	0	0



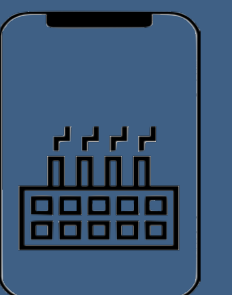
Dynamic Programming - House Robber

Problem Statement:

- Given N number of houses along the street with some amount of money
- Adjacent houses cannot be stolen
- Find the maximum amount that can be stolen

Bottom Up Approach

```
houseRobberBU(houses, currentIndex)
    tempAr = [0]*(length(houses)+2)
    for i in length(houses)-1 until -1
        tempAr[i] = max(houses[i]+tempAr[i+2], tempAr[i+1])
    return tempAr[0]
```



Dynamic Programming - Convert String

Problem Statement:

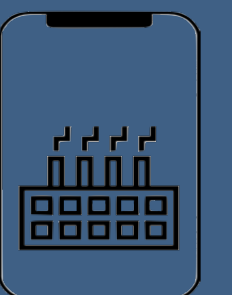
- S1 and S2 are given strings
- Convert S2 to S1 using delete, insert or replace operations
- Find the minimum count of edit operations

Example 1

- S1 = "catch"
- S2 = "carch"
- Output = 1
- Explanation : Replace "r" with "t"

Example 2

- S1 = "table"
- S2 = "tbres"
- Output = 3
- Explanation : Insert "a" to second position, replace "r" with "l" and delete "s"

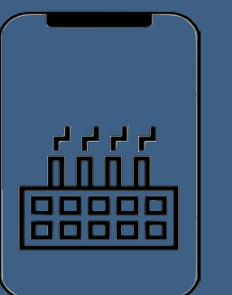


Dynamic Programming - Convert String

Problem Statement:

- S1 and S2 are given strings
- Convert S2 to S1 using delete, insert or replace operations
- Find the minimum count of edit operations

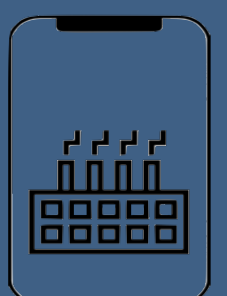
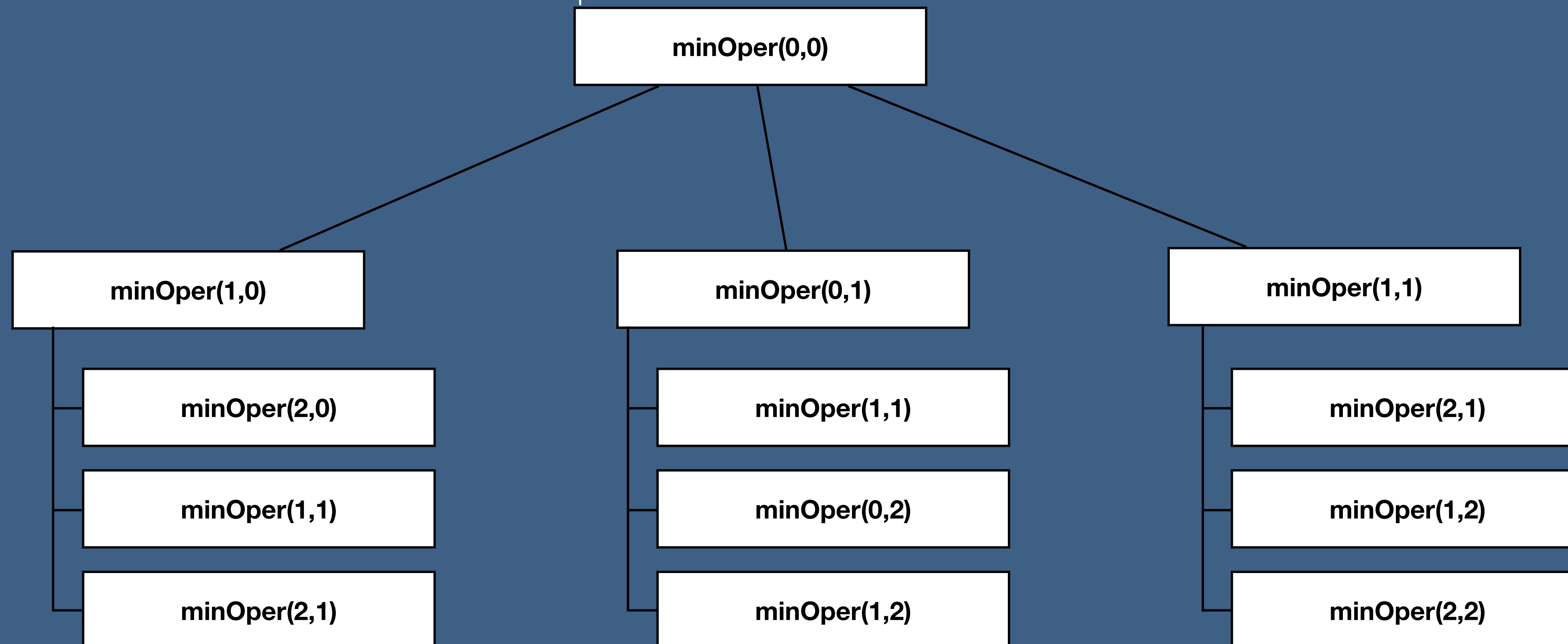
```
findMinOperation(s1, s2, index1, index2):  
    If index1 == len(s1)  
        return len(s2)-index2  
    If index2 == len(s2)  
        return len(s1)-index1  
    If s1[index1] == s2[index2]  
        return findMinOperation(s1, s2, index1+1, index2+1)  
  
    Else  
        deleteOp = 1 + findMinOperation(s1, s2, index1, index2+1)  
        insertOp = 1 + findMinOperation(s1, s2, index1+1, index2)  
        replaceOp = 1 + findMinOperation(s1, s2, index1+1, index2+1)  
        return min(deleteOp, insertOp, replaceOp)
```



Dynamic Programming - Convert String

Problem Statement:

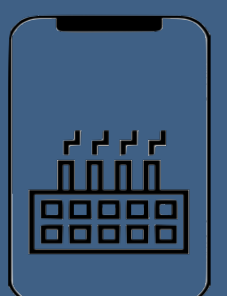
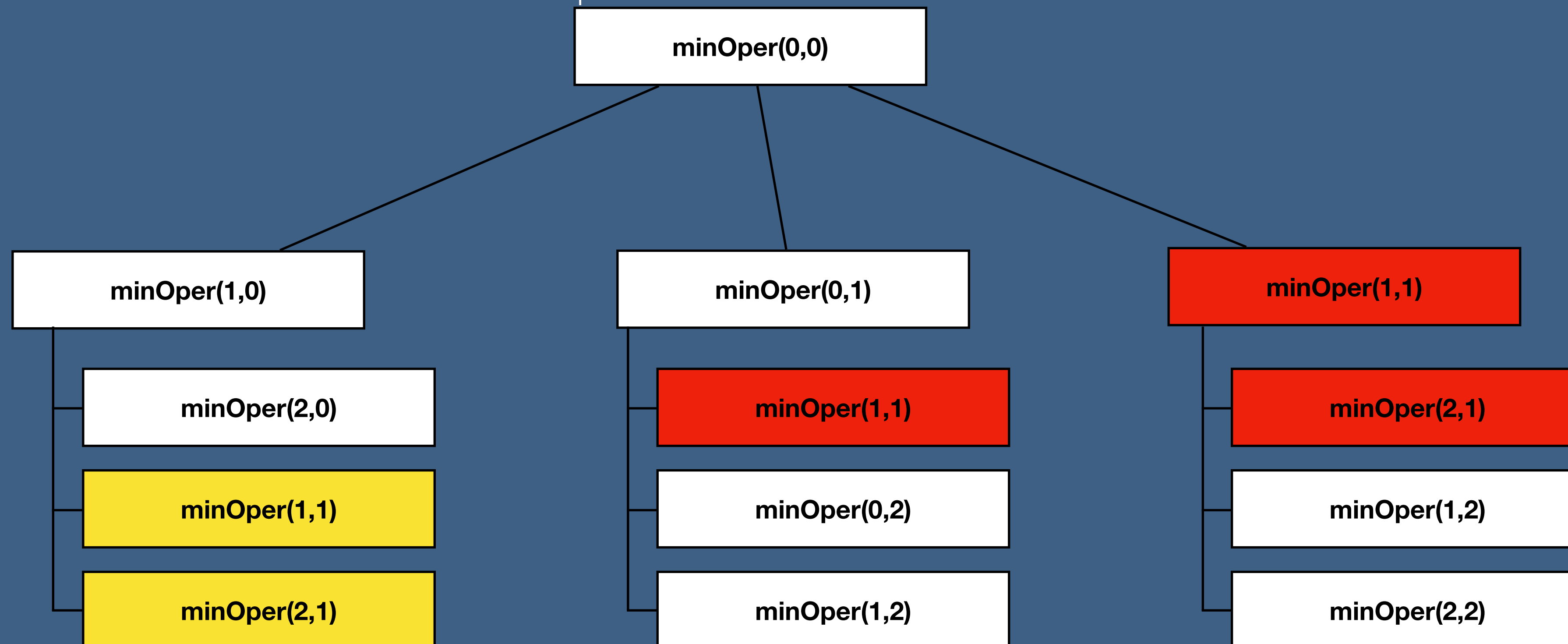
- S1 and S2 are given strings
- Convert S2 to S1 using delete, insert or replace operations
- Find the minimum count of edit operations



Dynamic Programming - Convert String

Problem Statement:

- S1 and S2 are given strings
- Convert S2 to S1 using delete, insert or replace operations
- Find the minimum count of edit operations



Dynamic Programming - Convert String

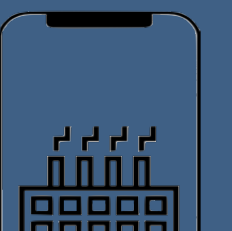
Problem Statement:

- S1 and S2 are given strings
- Convert S2 to S1 using delete, insert or replace operations
- Find the minimum count of edit operations

Top Down Approach

```
findMinOperation(s1, s2, index1, index2, dp[][])  
    if index1 == s1.length  
        return s2.length-index2  
    if index2 == len(s2)  
        return len(s1)-index1  
    if s1[index1] == s2[index2]  
        return findMinOperation(s1, s2, index1+1, index2+1, tempDict)  
    else:  
        dictKey = str(index1)+str(index2) —————→ Step 2  
        if dictKey not in tempDict:  
            deleteOp = 1 + findMinOperation(s1, s2, index1, index2+1, tempDict)  
            insertOp = 1 + findMinOperation(s1, s2, index1+1, index2, tempDict)  
            replaceOp = 1 + findMinOperation(s1, s2, index1+1, index2+1, tempDict)  
            tempDict[dictKey] = min(deleteOp, insertOp, replaceOp) —————→ Step 3  
        return tempDict[dictKey] —————→ Step 4
```

```
if(dp[i1][i2] == null) {  
    if(i1 == s1.length()) // if we have reached the end of s1, then insert all the remaining characters of s2  
        dp[i1][i2] = s2.length() - i2;  
  
    else if(i2 == s2.length()) // if we have reached the end of s2, then delete all the remaining characters of s1  
        dp[i1][i2] = s1.length() - i1;  
  
    else if(s1.charAt(i1) == s2.charAt(i2)) // If the strings have a matching character, recursively match for the remaining lengths.  
        dp[i1][i2] = findMinOperationsTD(dp, s1, s2, i1+1, i2+1);  
}
```



Dynamic Programming - Convert String

Problem Statement:

- S1 and S2 are given strings
- Convert S2 to S1 using delete, insert or replace operations
- Find the minimum count of edit operations

Bottom Up Approach

```
def findMinOperationBU(s1, s2, tempDict):
    for i1 in range(len(s1)+1):
        dictKey = str(i1)+'0'
        tempDict[dictKey] = i1
    for i2 in range(len(s2)+1):
        dictKey = '0'+str(i2)
        tempDict[dictKey] = i2

    for i1 in range(1, len(s1)+1):
        for i2 in range(1, len(s2)+1):
            if s1[i1-1] == s2[i2-1]:
                dictKey = str(i1)+str(i2)
                dictKey1 = str(i1-1)+str(i2-1)
                tempDict[dictKey] = tempDict[dictKey1]
            else:
                dictKey = str(i1)+str(i2)
                dictKeyD = str(i1-1)+str(i2)
                dictKeyI = str(i1)+str(i2-1)
                dictKeyR = str(i1-1)+str(i2-1)
                tempDict[dictKey] = 1 + min(tempDict[dictKeyD], min(tempDict[dictKeyI], tempDict[dictKeyR]))
    dictKey = str(len(s1))+str(len(s2))
    return tempDict[dictKey]
```

